Which team can damage the other? Which team is immune to the other? The dependency structure tells the story (Figure 17.6). The source code of ReSharper depends on the source code of Visual Studio. Thus there is nothing that the ReSharper team can do to disturb the Visual Studio team. But the Visual Studio team could completely disable the ReSharper team if they so desired.

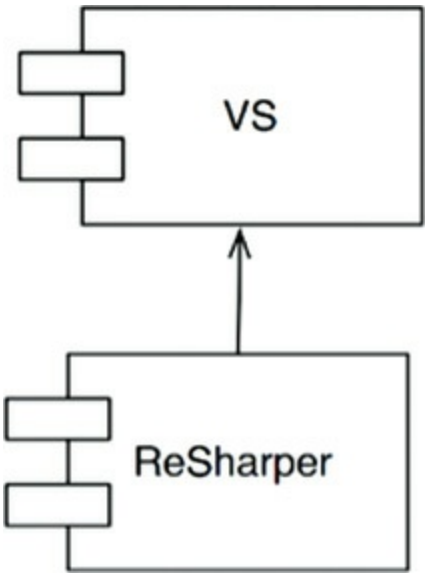

Figure 17.6 ReSharper depends on Visual Studio

That's a deeply asymmetric relationship, and it is one that we desire to have in our own systems. We want certain modules to be immune to others. For example, we don't want the business rules to break when someone changes the format of a web page, or changes the schema of the database. We don't want changes in one part of the system to cause other unrelated parts of the system to break. We don't want our systems to exhibit that kind of fragility.

Arranging our systems into a plugin architecture creates firewalls across which changes cannot propagate. If the GUI plugs in to the business rules, then changes in the GUI cannot affect those business rules.

Boundaries are drawn where there is an *axis of change*. The components on one side of the boundary change at different rates, and for different reasons, than the components on the other side of the boundary.

GUIs change at different times and at different rates than business rules, so there should be a boundary between them. Business rules change at different times and for different reasons than dependency injection frameworks, so there should be a boundary between them.

This is simply the Single Responsibility Principle again. The SRP tells us where to draw our boundaries.

CONCLUSION

To draw boundary lines in a software architecture, you first partition the system into components. Some of those components are core business rules; others are plugins that contain necessary functions that are not directly related to the core business. Then you arrange the code in those components such that the arrows between them point in one direction—toward the core business.

You should recognize this as an application of the Dependency Inversion Principle and the Stable Abstractions Principle. Dependency arrows are arranged to point from lower-level details to higher-level abstractions.

1. The word “architecture” appears in quotes here because three-tier is not an architecture; it’s a topology. It’s exactly the kind of decision that a good architecture strives to defer.
2. Many years later we were able to slip the Velocity framework into `FitNesse`.

18
BOUNDARY ANATOMY

The architecture of a system is defined by a set of software components and the boundaries that separate them. Those boundaries come in many different forms. In this chapter we'll look at some of the most common.

BOUNDARY CROSSING

At runtime, a boundary crossing is nothing more than a function on one side of the boundary calling a function on the other side and passing along some data. The trick to creating an appropriate boundary crossing is to manage the source code dependencies.

Why source code? Because when one source code module changes, other source

code modules may have to be changed or recompiled, and then redeployed. Managing and building firewalls against this change is what boundaries are all about.

THE DREADED MONOLITH

The simplest and most common of the architectural boundaries has no strict physical representation. It is simply a disciplined segregation of functions and data within a single processor and a single address space. In a previous chapter, I called this the source-level decoupling mode.

From a deployment point of view, this amounts to nothing more than a single executable file—the so-called monolith. This file might be a statically linked C or C++ project, a set of Java class files bound together into an executable jar file, a set of .NET binaries bound into a single `.EXE` file, and so on.

The fact that the boundaries are not visible during the deployment of a monolith does not mean that they are not present and meaningful. Even when statically linked into a single executable, the ability to independently develop and marshal the various components for final assembly is immensely valuable.

Such architectures almost always depend on some kind of dynamic polymorphism[1] to manage their internal dependencies. This is one of the reasons that object-oriented development has become such an important paradigm in recent decades. Without OO, or an equivalent form of polymorphism, architects must fall back on the dangerous practice of using pointers to functions to achieve the appropriate decoupling. Most architects find prolific use of pointers to functions to be too risky, so they are forced to abandon any kind of component partitioning.

The simplest possible boundary crossing is a function call from a low-level client to a higher-level service. Both the runtime dependency and the compile-time dependency point in the same direction, toward the higher-level component.

In Figure 18.1, the flow of control crosses the boundary from left to right. The `Client` calls function `f()` on the `Service`. It passes along an instance of `Data`. The `<DS>` marker simply indicates a data structure. The `Data` may be passed as a function argument or by some other more elaborate means. Note that the definition of the `Data` is on the *called* side of the boundary.

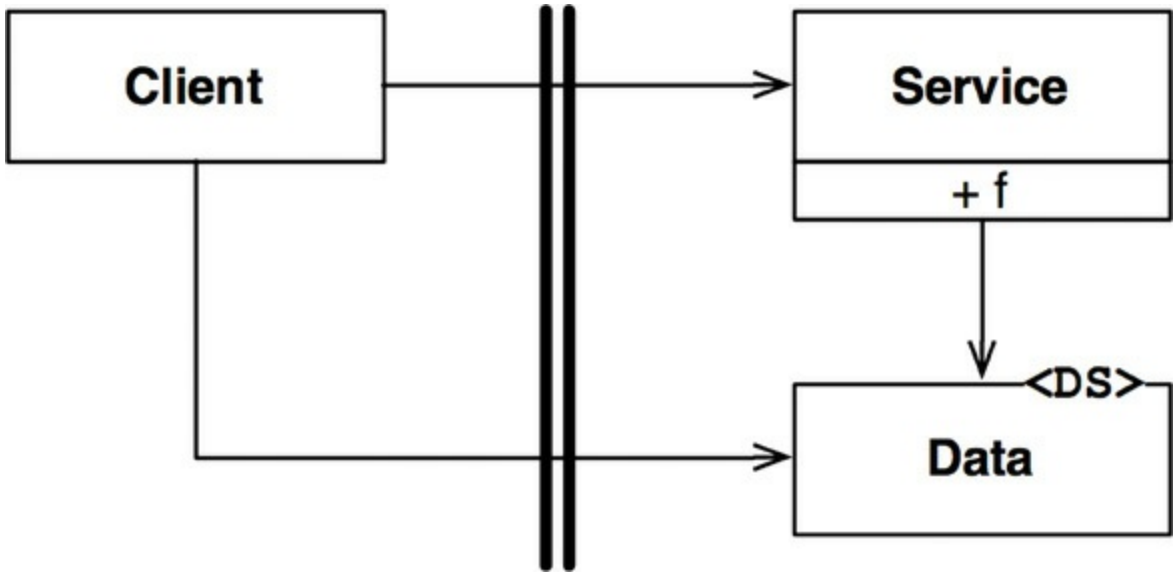

Figure 18.1 Flow of control crosses the boundary from a lower level to a higher level

When a high-level client needs to invoke a lower-level service, dynamic polymorphism is used to invert the dependency against the flow of control. The runtime dependency opposes the compile-time dependency.

In Figure 18.2, the flow of control crosses the boundary from left to right as before. The high-level `Client` calls the `f()` function of the lower-level `ServiceImpl` through the `Service` interface. Note, however, that all dependencies cross the boundary from right to left *toward the higher-level component*. Note, also, that the definition of the data structure is on the calling side of the boundary.

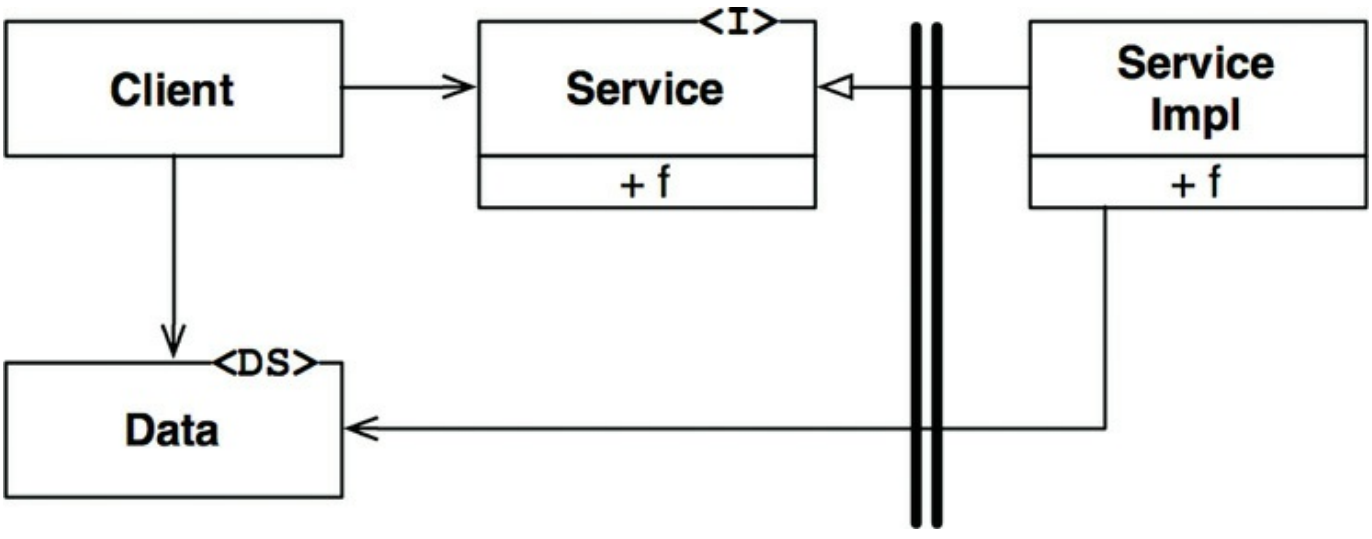

Figure 18.2 Crossing the boundary against the flow of control

Even in a monolithic, statically linked executable, this kind of disciplined

partitioning can greatly aid the job of developing, testing, and deploying the project. Teams can work independently of each other on their own components without treading on each other's toes. High-level components remain independent of lower-level details.

Communications between components in a monolith are very fast and inexpensive. They are typically just function calls. Consequently, communications across source-level decoupled boundaries can be very chatty.

Since the deployment of monoliths usually requires compilation and static linking, components in these systems are typically delivered as source code.

DEPLOYMENT COMPONENTS

The simplest physical representation of an architectural boundary is a dynamically linked library like a .Net DLL, a Java jar file, a Ruby Gem, or a UNIX shared library. Deployment does not involve compilation. Instead, the components are delivered in binary, or some equivalent deployable form. This is the deployment-level decoupling mode. The act of deployment is simply the gathering of these deployable units together in some convenient form, such as a WAR file, or even just a directory.

With that one exception, deployment-level components are the same as monoliths. The functions generally all exist in the same processor and address space. The strategies for segregating the components and managing their dependencies are the same.[2]

As with monoliths, communications across deployment component boundaries are just function calls and, therefore, are very inexpensive. There may be a one-time hit for dynamic linking or runtime loading, but communications across these boundaries can still be very chatty.

THREADS

Both monoliths and deployment components can make use of threads. Threads are not architectural boundaries or units of deployment, but rather a way to organize the schedule and order of execution. They may be wholly contained within a component, or spread across many components.